

GITHUB COPILOT - DEEP DIVE

Apply GitHub Copilot Capabilities

- From a single suggestion to a full conversation with Copilot Chat
- Widening what Copilot can see, and choosing your model deliberately
- Handing multi-file work to Agent mode
- Teaching Copilot your team's stable conventions, once
- Setting up your own capstone project for the rest of the course



Chat — Slash Commands and Context Variables

- Copilot Chat: a conversational front end that reads more than the cursor position
- Slash commands say what kind of task this is — /explain, /fix, /fixTestFailure, /tests, /new
- Context variables (#) say exactly what to read — #file, #selection, #function, #class, #project
- Chat participants (@) reach domain-specific help — @terminal, @github
- A slash command plus a context variable beats plain language, same as naming a pattern in Module 1

```
/explain #file:IngestHandler.java
```

slash command

context variable

A slash command names the task; a context variable names the material.

Widening What a Single Question Can See

- Chat can reference a whole method, class, or the entire open solution
- It shows the sources it actually used — check "did it read the real code" directly
- Chat sessions are organized into threads
- A fresh thread per unrelated task keeps what's being read focused (Module 1 callback)



Sources Used

IngestHandler.java
TelemetryReading.java

Chat Beyond Text — Images, Voice, and the Terminal



Vision

Attach a screenshot or sketch directly to a prompt



Voice

Availability varies by IDE — verify before demonstrating



Terminal (CLI)

Same assistant, terminal-native — deploys, log inspection

Voice input's exact mechanism and availability aren't independently verified for this course — confirm before demonstrating.

Two Shortcuts Worth Knowing

- Inline Editor Chat — anchored to a specific spot in the file you're editing
- Quick Chat — a fast, throwaway question, no persistent thread needed
- Match the shortcut to Module 1's "two-second glance vs. sit-down explanation" test



Inline Editor Chat

Anchored to a specific spot in the file you're editing



Quick Chat

Fast, throwaway — no persistent thread

Which Model Is Actually Answering

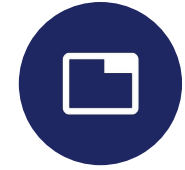
- Copilot Chat supports more than one underlying model, selectable per session
- Models trade off speed, reasoning depth, and cost differently
- Deep reasoning over a large context vs. a fast, cheap answer — a real choice, not a default



A real trade-off, not a default

[Verify the current model list and their individual trade-offs against docs.github.com before presenting specifics.]

Problem — It Doesn't Understand My Codebase



- A common complaint, but rarely the model actually being bad
- "Context-aware" means: the open file, related files, recent edits, and whatever's attached
- Everything retrieved shares one fixed-size context window, measured in tokens
- Think of it as a fixed number of open tabs on one shared workspace

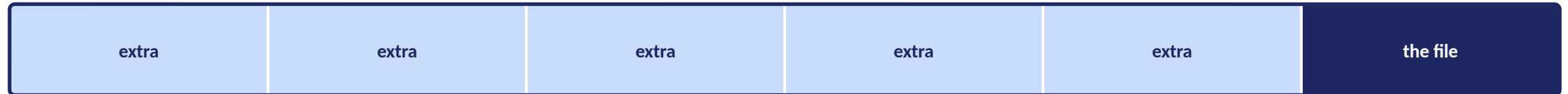


Adding one more thing closes something else — whether you asked for that or not.

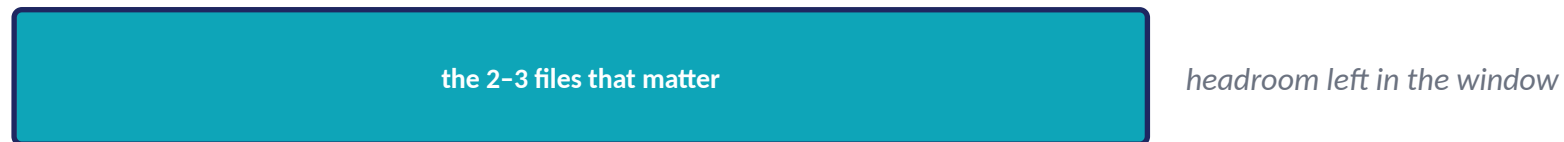
Solution — Attach Deliberately, Not “To Be Safe”

- Attaching every file "to be safe" doesn't add understanding
- It spends the same fixed budget on material the task doesn't need
- That's at the direct expense of the two or three files that actually matter
- Per-user, per-repository, and per-organization settings decide what's available to read at all

Attaching "to be safe"



Attaching deliberately

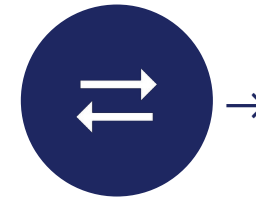


From One File to Several — Agent Mode

- You hand it a goal; it plans, uses tools, and acts across multiple files
- It iterates on its own when it hits an error, a failing test, or a build break
- Stays synchronous — you watch, review the plan and diffs, decide what's actually done
- Not the coding agent, which works asynchronously without a human watching
- Multi-file plan-and-execute → Agent mode; a single question → plain Chat



Plan



Act Across Files



Review Diff

HANDS-ON Explaining and Modifying Code with Chat and Agent Mode

Teaching Copilot Your Team's Stable Conventions, Once

- Custom instructions turn stable, repeated context into something Copilot reads automatically
- Repository-wide: `.github/copilot-instructions.md`, ordinary Markdown, genuinely stable conventions
- Path-specific: `NAME.instructions.md` under `.github/instructions/`, scoped by a required `applyTo` glob
- A file matching both gets both together — not one overriding the other



Repository-wide

`.github/copilot-instructions.md` — ordinary Markdown, stable conventions only



Path-specific

`NAME.instructions.md`, scoped by a required `applyTo` glob pattern

HANDS-ON Writing a Custom Instructions File

Setting Up Your Own Capstone Project



Vehicle Telemetry Visualization

Fleet-scale telemetry ingest and dashboards



Digital Twin Assembly-Line Simulation

A simulated line, mirrored in real time



Shop-Floor Resource Allocation

Scheduling people and equipment efficiently

Each team is assigned or chooses one — confirm starter-repo access, sign Copilot in, write a one-sentence MVP scope. Scoped and readied here, not built on until Module 3.

HANDS-ON Capstone Project Kickoff and Setup